

11 — EXERCISES ON FILE STREAMS

Quick reference

```
#include <fstream>
#include <string>

// --- Writing ---
std::ofstream out("notes.txt"); // open for writing (overwrites)
if (!out) { /* file could not be opened */ }
out << "hello " << 42 << "\n"; // same << syntax as std::cout
out.close(); // optional but good practice

// --- Reading word by word (operator >>) ---
std::ifstream in("notes.txt");
if (!in) { /* file could not be opened */ }
std::string word;
int number;
in >> word >> number; // reads "hello" then 42
// >> skips whitespace and newlines

// --- Reading line by line (std::getline) ---
std::string line;
while (std::getline(in, line)) { // returns false at end of file
    // process line
}
```

Key points:

- Always check that a file opened successfully with `if (!stream)`.
- `>>` skips leading whitespace and stops at the next whitespace character — it reads one *token* at a time.
- `std::getline` reads everything up to (and discarding) the next `'\n'`, including spaces.
- After `>>` reads the last token on a line, a stray `'\n'` may remain in the buffer. Call `in.ignore()` before `std::getline` to discard it.

Part A — Writing to a file

Problem 1 — What ends up in the file?

For each program, write the exact content that would appear in the output file.
Practise № 11

Writing numbers

```
1 #include <fstream>
2
3 int main() {
4     std::ofstream out("track.txt");
5     out << "title: " << "Echoes" << "\n";
6     out << "bpm: " << 120 << "\n";
7     out << "bars: " << 4 * 16 << "\n";
8     out.close();
9     return 0;
10 }
```

Content of track.txt:

Writing in a loop

```
1 #include <fstream>
2
3 int main() {
4     std::ofstream out("pixels.txt");
5     int values[4] = {10, 50, 200, 80};
6     for (int i = 0; i < 4; i = i + 1)
7         out << i << " " << values[i] << "\n";
8     return 0;
9 }
```

Content of pixels.txt:

Problem 2 — Spot the bug

This program should write five scores to `scores.txt`, one per line. It has two bugs.

Buggy writer

```
1 #include <fstream>
2
```

```

3  int main() {
4      std::ofstream out;
5
6      int scores[5] = {72, 88, 45, 95, 60};
7      for (int i = 0; i < 5; i = i + 1)
8          out >> scores[i] >> "\n";
9
10     return 0;
11 }

```

Bug 1:

Fix 1:

Bug 2:

Fix 2:

Problem 3 — Write the code

Write a complete program that takes a filename and a list of integers from the command line and writes each integer on its own line to that file.

Invocation:

```
./write notes.txt 60 62 64 65 67
```

Expected content of `notes.txt`:

```

60
62
64
65
67

```

If the file cannot be opened, print `"error: could not open file"` and return 1.

Part B — Reading with >>

Problem 1 — Trace the reads

`track.txt` contains:

```
Echoes 120 64
```

What does this program print?

Reading tokens

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4
5  int main() {
6      std::ifstream in("track.txt");
7      std::string title;
8      int bpm, bars;
9
10     in >> title >> bpm >> bars;
11
12     std::cout << title << std::endl;    // line A
13     std::cout << bpm << std::endl;      // line B
14     std::cout << bars << std::endl;     // line C
15     std::cout << bpm * bars << std::endl; // line D
16
17     return 0;
18 }

```

Line A:

Line B:

Line C:

Line D:

Problem 2 — Reading multiple lines with >>

pixels.txt contains (as written by exercise 1.1):

```

0 10
1 50
2 200
3 80

```

What does this program print?

Summing pixel values

```

1  #include <iostream>
2  #include <fstream>
3
4  int main() {
5      std::ifstream in("pixels.txt");
6      int idx, value, total = 0, count = 0;
7
8      while (in >> idx >> value) {

```

```

9         total = total + value;
10        count = count + 1;
11    }
12
13    std::cout << count << std::endl;    // line A
14    std::cout << total << std::endl;    // line B
15    std::cout << total / count << std::endl; // line C
16
17    return 0;
18 }

```

Line A:

Line B:

Line C:

What does `while (in >> idx >> value)` actually test? When does it become false?

Problem 3 — Spot the bug

This program reads a file of the form `<name> <score>` per line and should print the name and score of each entry. It has one bug.

results.txt:

```

Alice 88
Bob 74
Clara 95

```

Buggy reader

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4
5  int main() {
6      std::ifstream in("results.txt");
7      if (!in) {
8          std::cout << "error" << std::endl;
9          return 1;
10     }
11
12     std::string name;
13     int score;

```

```

14
15     while (in >> score >> name) {
16         std::cout << name << ": " << score << std::endl;
17     }
18
19     return 0;
20 }

```

Bug:

Fix:

What does the buggy version actually read into each variable on the first iteration?

Part C — Reading with `std::getline`

Problem 1 — `>>` vs `getline`: what do they read?

`song.txt` contains:

```

Miles Davis
Kind of Blue
1959

```

For each program, write what is stored in each variable after the reads.

Version A (using `>>`):

```

std::ifstream in("song.txt");
std::string a, b;
int c;
in >> a >> b >> c;

```

a =

b =

c =

Why does `a` not contain the full first line?

Version B (using `getline`):

```

std::ifstream in("song.txt");
std::string a, b, c;
std::getline(in, a);
std::getline(in, b);
std::getline(in, c);

```

a =

b =

c =

Problem 2 — Trace: reading a whole file line by line

setlist.txt contains:

```
So What
Freddie Freeloader
Blue in Green
All Blues
Flamenco Sketches
```

What does this program print?

Line counter

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4
5  int main() {
6      std::ifstream in("setlist.txt");
7      std::string line;
8      int count = 0;
9
10     while (std::getline(in, line)) {
11         count = count + 1;
12         std::cout << count << ": " << line << std::endl;
13     }
14
15     std::cout << "total: " << count << std::endl;
16     return 0;
17 }
```

Output:

Problem 3 — Mixing >> and getline

tracks.txt contains:

```
3
So What
Freddie Freeloader
Blue in Green
```

The first line is the number of tracks; the rest are track names (which may contain spaces).
What does this program print?

Mixed read

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4
5  int main() {
6      std::ifstream in("tracks.txt");
7      int n;
8      in >> n;
9      in.ignore();           // discard the '\n' after the number
10
11     std::cout << n << std::endl; // line A
12
13     std::string name;
14     for (int i = 0; i < n; i = i + 1) {
15         std::getline(in, name);
16         std::cout << i + 1 << ": " << name << std::endl;
17     }
18
19     return 0;
20 }
```

Output:

Try removing `in.ignore()`. What happens then?

Part D — Putting it all together

Problem 1 — Write the code: pixel file statistics

`image.txt` contains one integer per line (pixel brightness values, 0–255). The number of lines is not known in advance.

Write a complete program that:

1. Takes the filename as `argv[1]`; prints an error and returns 1 if the argument is missing or the file cannot be opened.
2. Reads all pixel values from the file.
3. Prints the count, sum, minimum, and maximum.

Given `image.txt`:

```
34
210
88
5
175
120
```

Expected output:

```
count: 6
sum: 632
min: 5
max: 210
```

Problem 2 — Write the code: setlist filter

Write a program that:

1. Takes two filenames from `argv`: an input file and an output file.
2. Reads the input line by line. Each line has the format `<title> <duration>` where `<title>` is a single word and `<duration>` is an integer (seconds).
3. Writes to the output file only the lines whose duration is strictly above 200 seconds, preserving the same format.
4. After writing, prints the number of songs kept.

Given setlist.txt:

```
Echoes 340  
Shine 195  
Breathe 163  
Time 252  
Money 382
```

Expected content of output file:

```
Echoes 340  
Time 252  
Money 382
```

Expected terminal output:

```
3 songs kept
```